



Plan du cours



Chap 1 : Initiation au Langage Java

- Introduction
- Outils de développement
- Données, Types et Operateurs
- Les structures alternatives et répétitives
- Les tableaux de données et chaînes de caractères
- Les instructions d'écriture et de lecture

Chap 2 : Java Orienté Objet

- Classes et Objets
- Héritage
- Packages
- Classes internes et Interfaces
- Les exceptions

Chap 3 : Introduction aux IHMs

- Evolution des IHM dans le temps
- Ergonomie
- Le patron d'architecture logicielle MVC

Chap 4 : Eléments de base d'une IHM Java

- Composants de base
- Composants SWING
- Les conteneurs
- Les composants atomiques
- Evènements & Ecouteurs
- Principes programmation événementielle
- Listeners java
- Gestionnaires de positionnement

Chap 5 : Connexion à une base de données

IHM - Java

2

Connexion à une base de données

(CHAPITRE 5)



L'API JDBC et les bases de données :

Dans une application JAVA utilisant une base de données avec L'API JDBC, on trouvera généralement les étapes suivantes :

- Importer les packages pour les classes JDBC: `import java.sql.*;`
- Enregistrer le pilote JDBC
- Ouvrir la connexion à la base de données
- Émission de requêtes SQL vers la base
- Réception et traitement des résultats de ces requêtes
- Fermeture de la connexion
- Nous allons suivre ces étapes sur un exemple
 - Nous considérons une base de données MySQL appelée `g_personnel`

Connexion à une base de données

(CHAPITRE 5)



Préparer la base de données :

Télécharger MySQL

General Availability (GA) Releases Archives

MySQL Installer 8.0.28

Select Operating System:
Microsoft Windows

Looking for previous GA versions?

Operating System	Version	Size	Action
Windows (x86, 32-bit), MSI Installer (mysql-installer-web-community-8.0.28.0.msi)	8.0.28	2.3M	Download
Windows (x86, 32-bit), MSI Installer (mysql-installer-community-8.0.28.0.msi)	8.0.28	435.7M	Download

MD5: 514567a7503998d271a20b86057f15d0 | Signature

MD5: ef223cbd7873a057d5aa0ebcf596201 | Signature

We suggest that you use the MD5 checksums and GnuPG signatures to verify the integrity of the packages you download.

<https://dev.mysql.com/downloads/windows/installer/8.0.html>

Connexion à une base de données

(CHAPITRE 5)



Préparer la base de données :

--Code SQL:

```
CREATE DATABASE g_personnel;
USE g_personnel;
CREATE TABLE IF NOT EXISTS `Personne` (
  `idPersonne` TINYINT,
  `nom` varchar(15) default NULL,
  `prenom` varchar(15) default NULL,
  `age` TINYINT,
  PRIMARY KEY (`idPersonne`)
)
INSERT INTO `Personne` VALUES(1, 'ALAQUI', 'Mohammed', 22);
INSERT INTO `Personne` VALUES(2, 'HACHIMI', 'Fatima', 21);
INSERT INTO `Personne` VALUES(3, 'NOUACH', 'Adil', 23);

SELECT * FROM g_personnel.Personnel
```

IHM - Java

5

Connexion à une base de données

(CHAPITRE 5)



A- L'étape de connexion:

Avant l'exécution du programme un pilote de connexion (Connector) doit être ajouté au projet en cours

Télécharger le pilote **jdbc** (si ce n'est déjà fait)

<https://dev.mysql.com/downloads/connector/j/5.0.html>

Installation du pilote JDBC sous Eclipse

- Télécharger le fichier **mysql-connector-java-5.1.48.tar** dans un répertoire choisi.
- Décompresser l'archive
- Sous Eclipse cliquez avec le bouton droit sur votre projet
- Ouvrir **Propreties** puis **Java Build Path** puis **Libraries**
- Cliquez sur le bouton **Add External JARs**
- Ouvrir le fichier JAR : **mysql-connector-java-5.1.48-bin**

IHM - Java

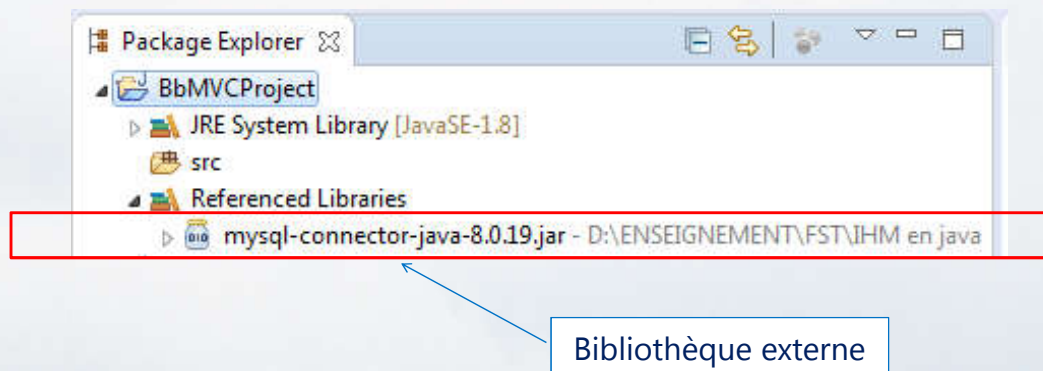
6

Connexion à une base de données

(CHAPITRE 5)



A- L'étape de connexion:



IHM - Java

7

Connexion à une base de données

(CHAPITRE 5)



A- L'étape de connexion:

1- Chargement du pilote JDBC

Pour exploiter une base de données, une application Java doit d'abord charger le pilote qui va servir à la connexion.

```
try {
    //Chargement de pilote
    pilote=new String("com.mysql.jdbc.Driver");
    Class.forName(pilote);
    //...
}
catch( ClassNotFoundException e) {
    System.err.println("Erreur lors du chargement du pilote : " + e);
}
```

IHM - Java

8

Connexion à une base de données

(CHAPITRE 5)



A- L'étape de connexion:

2- Connexion à la base

Après le chargement du pilote la connexion à la base devient possible via la méthode :

Connection myCnx=java.sql.DriverManager.getConnection(String url, String user, String password) **throws SQLException**

Avec:

- **DriverManager** : classe Java détenant la liste des pilotes disponibles pour l'application
- **Connection** : classe Java établissant un lien entre l'application et la base grâce auquel l'application va transmettre des requêtes SQL à la base et recevoir des résultats
- **url** : nom identifiant la base de données.
- **user** : Nom de l'utilisateur
- **password** : Mot de passe

Connexion à une base de données

(CHAPITRE 5)



A- L'étape de connexion:

2- Connexion à la base

URL a la forme suivante :

jdbc:mysql://[machine][:port]/base[?param1=val1][¶m1=val2] ...

Où

- **jdbc:mysql** : Le nom du protocole employé
- **[machine][:port]** Le nom de la machine suivi de manière optionnelle d'un numéro de port (indispensable si MySQL a été configuré pour utiliser un port différent du port par défaut).
- **base** Le nom de la BDD à laquelle nous souhaitons accéder.
- **param1=val1,param1=val2,etc.,** Des paires (**nom,valeur**) de paramètres passées au serveur

```
url = new String("jdbc:mysql://localhost:8080/g_personnel");
```

Connexion à une base de données

(CHAPITRE 5)



A- L'étape de connexion:

```
import java.sql.*;
public class Connexion {
    Connection myCnx ;
    String url,pilote;
    public Connexion() {
        try {
            //Chargement de pilote
            pilote=new String("com.mysql.cj.jdbc.Driver");
            Class.forName(pilote);
            // Definition de l'URL de connexion
            url = new String("jdbc:mysql://127.0.0.1/g_personnel?serverTimezone=UTC");
            // Etablissement de la Connexion
            myCnx = DriverManager.getConnection(url,"root","");
            System.out.println("CONNEXION ETABLIE");
            // Fermeture de la connexion
            myCnx.close();
        }
        catch( ClassNotFoundException e) {System.err.println("Erreur lors du chargement
        du pilote : " + e); }
        catch (SQLException e) {System.err.println("Erreur de syntaxe SQL : " + e);}
    }
    public static void main(String[] args) {
        new Connexion();
    }
}
```

11

Connexion à une base de données

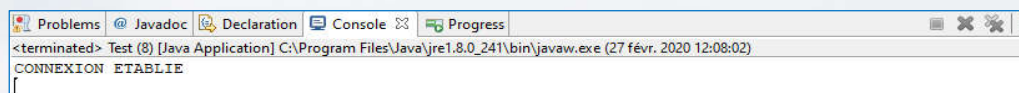
(CHAPITRE 5)



A- L'étape de connexion:

2- Connexion à la base

```
// Definition de l'URL de connexion
url = new String("jdbc:mysql://localhost/g_personnel?serverTimezone=UTC&useSSL=false");
```



IHM - Java

12

Connexion à une base de données

(CHAPITRE 5)



B- Émission de requêtes SQL vers la base:

L'interface JDBC permet :

- l'émission de requêtes SQL vers la base de données connectée à l'application Java
- le traitement des résultats de ces requêtes.

Le langage SQL (Structured Query Language) est un langage de requêtes standardisé pour les bases de données relationnelles.

On y distingue plusieurs types de requêtes :

- les requêtes d'interrogation de la base LMD (**SELECT**)
- les requêtes de mise à jour de la base LMD (**INSERT, DELETE, UPDATE**)
- les requêtes de création/destruction/modification de tables LDD (**CREATE, DROP, ALTER**)

Connexion à une base de données

(CHAPITRE 5)



B- Émission de requêtes SQL vers la base:

1- La classe Statement

Pour émettre une requête SQL vers une base de données, l'application Java doit disposer d'un objet de type **Statement**.

- Cet objet stockera le texte de la requête.
- Il est nécessairement lié à la connexion en cours.

Si **myCnx** est l'objet symbolisant la connexion avec la base de données, un objet **Statement** est obtenu de la façon suivante :

```
Statement st = myCnx.createStatement();
```

Une fois obtenu un objet **Statement**, on peut émettre des requêtes SQL.

Cela se fera différemment selon que la requête est une requête d'interrogation ou de mise à jour de la base(LMD/LDD).

Connexion à une base de données

(CHAPITRE 5)



B- Émission de requêtes SQL vers la base:

2- Émettre une requête d'interrogation de la base

Une requête d'interrogation est classiquement une requête du type :

```
SELECT col1, col2,... FROM table1, table2,...
WHERE condition
ORDER BY expression ...
```

1. Une jointure est faite avec toutes les tables qui sont derrière le mot clé **FROM**
2. Seules les colonnes qui sont derrière le mot clé **SELECT** sont conservées
3. Seules les lignes vérifiant la condition du mot clé **WHERE** sont conservées
4. Les lignes résultantes ordonnées selon l'expression du mot clé **ORDER BY** forment le résultat de la requête.

IHM - Java

15

Connexion à une base de données

(CHAPITRE 5)



B- Émission de requêtes SQL vers la base:

2- Émettre une requête d'interrogation de la base

Pour exécuter ce type de requête, la classe Statement offre la méthode :

```
ResultSet java.sql.Statement.executeQuery(String sql)
```

Exemples

La base **g_personnel** qu'on traite contient une table **personne** :

#	Nom	Type	Interclassement	Attributs	Null
☐ 1	idPersonne	tinyint(4)			Non
☐ 2	nom	varchar(15) latin1_swedish_ci			Oui
☐ 3	prenom	varchar(15) latin1_swedish_ci			Oui
☐ 4	age	tinyint(4)			Oui

idPersonne	nom	prenom	age
1	ALAOUI	Mohammed	22
2	HACHIMI	Fatima	21
3	NOUACH	Adil	23

IHM - Java

16

Connexion à une base de données

(CHAPITRE 5)



B- Émission de requêtes SQL vers la base:

2- Émettre une requête d'interrogation de la base

Pour afficher le contenu de la table **Personne**

```
//Création d'une instruction
Statement statement= myCnx.createStatement();

// Exécution d'une requete
String query = "select * from personne";
ResultSet rs = statement.executeQuery(query);
while(rs.next()) {
    System.out.println(
        rs.getString(1)+" "+
        rs.getString(2)+" "+
        rs.getString(3)+" "+
        rs.getString(4)
    );
}
```

IHM - Java

17

Connexion à une base de données

(CHAPITRE 5)



B- Émission de requêtes SQL vers la base:

2- Émettre une requête d'interrogation de la base

```
Console
<terminated> Connexion [Java Application] C:\Program Files\Java\jre1.8.0_45\bin\javaw.exe (30 mars 2022 à 22:27:11)
CONNEXION ETABLIE
idPersonne  nom    prenom  age
1           ALAQUI Mohammed 22
2           HACHIMI Fatima 21
3           NOUACH Adil 23
```

```
String query = "select * from personne where age <22";
```

```
Console
<terminated> Connexion [Java Application] C:\Program Files\Java\jre1.8.0_45\bin\javaw.exe (30 mars 2022 à 22:28:45)
CONNEXION ETABLIE
idPersonne  nom    prenom  age
2           HACHIMI Fatima 21
```

IHM - Java

18

Connexion à une base de données

(CHAPITRE 5)



B- Émission de requêtes SQL vers la base:

3- La classe **ResultSet** : résultat d'une requête select

Un objet de type **ResultSet** représente une **table**, c'est à dire un ensemble de lignes et de colonnes.

Lors de la création initiale du **ResultSet**, la ligne courante est la ligne n° 1 si le **ResultSet** n'est pas vide.

ResultSet dispose de plusieurs méthodes, on cite entre autres :

- **boolean next()** Permet d'exploiter les différents champs
- **String getString(int indiceColonne)**
- **String getString("nomColonne")**

Permettent d'obtenir les différents champs de la ligne courante

Connexion à une base de données

(CHAPITRE 5)



B- Émission de requêtes SQL vers la base:

4- La classe **ResultSetMetadata**

La classe **ResultSetMetadata** donne des informations sur la nature des colonnes de la table à savoir :

- le nombre de colonnes du **ResultSet** :
int getColumnCount()
- le nom de la colonne **i** du **ResultSet** (**i** ≥ 1)
String getColumnLabel(int i)
- Si **RS** est un **ResultSet**, le **ResultSetMetadata** associé est obtenu par :
RS.getMetaData()

Exemple d'utilisation :

```
System.out.println("Nombre de colonne dans cette base est " +  
rs.getMetaData().getColumnCount());
```

Connexion à une base de données

(CHAPITRE 5)



B- Émission de requêtes SQL vers la base:

```
Statement statement = con.createStatement();
String query = "select * from personne";
ResultSet rs = statement.executeQuery(query);
ResultSetMetaData rsMetaData = rs.getMetaData();
int numberOfColumns = rsMetaData.getColumnCount();
System.out.println("Nombre de colonne" + numberOfColumns);
for (int i = 1; i <= numberOfColumns; i++)
    System.out.print(rsMetaData.getColumnName(i)+" ");
while(rs.next()) {
    System.out.println(rs.getString(1)+" \t"+ rs.getString(2)+" \t"+
rs.getString(3)+"\t "+ rs.getString(4)) ;
}
```

Console

```
<terminated> Connexion [Java Application] C:\Program Files\Java\jre1.8.0_45\bin\javaw.exe (30 mars 2022 à 22:42:06)
CONNEXION ETABLIE
Nombre de colonne4
idPersonne nom prenom age
1 ALAOUI Mohammed 22
2 HACHIMI Fatima 21
3 NOUACH Adil 23
<terminated> Connexion [
```

IHM - Java

21

Connexion à une base de données

(CHAPITRE 5)



B- Émission de requêtes SQL vers la base:

5- Émettre une requête de mise à jour de la base de données

Un objet de type **Statement** permet de stocker les requêtes SQL.

La méthode que cet objet utilise pour émettre des requêtes SQL de mise à jour (**INSERT**, **UPDATE**, **DELETE**) est :

```
int java.sql.Statement.executeUpdate(String sql)
```

qui renvoie le nombre de lignes affectées par l'opération de mise à jour.

IHM - Java

22

Connexion à une base de données

(CHAPITRE 5)



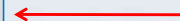
B- Émission de requêtes SQL vers la base:

5- Émettre une requête de mise à jour de la base de données

Ajouter une ligne :

```
String queryInsert="INSERT INTO personne VALUES(4, 'HASSAN', 'ELRADI',44)";
statement.executeUpdate(queryInsert);
```

idPersonne	1	nom	prenom	age
1	ALAOUI	Mohammed	22	
2	HACHIMI	Fatima	21	
3	NOUACH	Adil	23	
4	HASSAN	ELRADI	44	



Connexion à une base de données

(CHAPITRE 5)



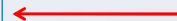
B- Émission de requêtes SQL vers la base:

5- Émettre une requête de mise à jour de la base de données

Modifier une ligne :

```
String queryUpdate="update personne set AGE=18 WHERE PRENOM='FATIMA'";
statement.executeUpdate(queryUpdate);
```

idPersonne	1	nom	prenom	age
1	ALAOUI	Mohammed	22	
2	HACHIMI	Fatima	18	
3	NOUACH	Adil	23	
4	HASSAN	ELRADI	44	



Connexion à une base de données

(CHAPITRE 5)



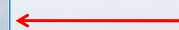
B- Émission de requêtes SQL vers la base:

5- Émettre une requête de mise à jour de la base de données

Supprimer une ligne :

```
String queryDelete="DELETE FROM personne where NOM='Alaoui'";
statement.executeUpdate(queryDelete);
```

idPersonne	1	nom	prenom	age
2	HACHIMI	Fatima	18	
3	NOUACH	Adil	23	
4	HASSAN	ELRADI	44	



IHM - Java

25

Connexion à une base de données

(CHAPITRE 5)



Exercice :

Soit la table Véhicule ci-dessous :

Programmer l'interface ci contre pour l'affichage et la navigation entre les véhicules enregistrés

VEHICULE	
nummma	
modele	
couleur	

IHM - Java

26